

ESERCIZIO 1 PRIMA PARTE

Un calcolatore ha un bus sincrono che funziona con una frequenza di 2.4 GHz. La capacità del bus dati è di 32 bit. Il trasferimento attraverso il bus richiede un ciclo di clock. La memoria ha un tempo di ciclo pari a tre cicli di clock.

Spiegando chiaramente il protocollo di lettura sincrono di una parola di memoria, precisando il numero minimo di segnali di controllo che occorre trasmettere sul bus, calcolare il tempo di lettura di una parola da 32 bit, considerando una memoria indirizzata con 32 bit.

$$A) T = \frac{1}{2,4 \cdot 10^9 \text{ Hz}} = 0,416 \text{ ns}$$

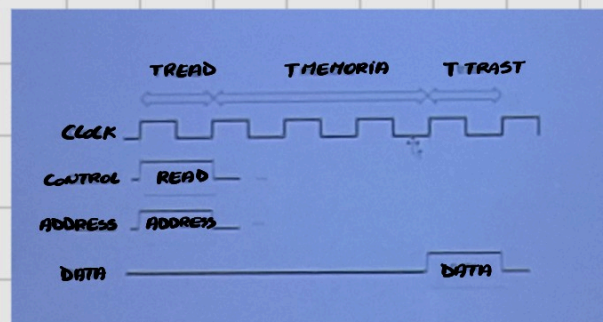
1 ciclo di clock $\Rightarrow 0,416 \text{ ns}$

3 cicli di clock $\Rightarrow 1,248 \text{ ns}$

③

Tempo di lettura = $(3 + 3) \times \frac{1}{2,4 \times 10^9}$ secondi
Tempo di lettura = $6 \times \frac{1}{2,4 \times 10^9}$ secondi
Tempo di lettura = $\frac{6}{2,4 \times 10^9}$ secondi
Tempo di lettura = $2,5 \times 10^{-9}$ secondi
Tempo di lettura = 2.5 ns

$(0,42 + 0,42 + 1,26) \text{ ns}$
 $2,10 \text{ ns}$



ESERCIZIO 2 SECONDA PARTE

Un calcolatore esegue 10^5 istruzioni/s. L'esecuzione di una istruzione richiede 5 cicli di clock, 3 dei quali tengono occupato il bus di I/O. Si ipotizzi che il 65% delle istruzioni sia usato dalla CPU per eseguire programmi che non contengono trasferimenti di I/O. L'ampiezza della linea dati del bus è pari a 32 bit. Si consideri il caso in cui il trasferimento di una parola da 32 bit fra una periferica e la memoria principale avvenga mediante I/O da programma per mezzo delle seguenti 4 istruzioni:

- LOAD parola dalla periferica al registro CPU
- STORE parola da registro CPU a memoria
- generazione indirizzo di memoria successivo
- conteggio dati da trasferire.

Calcolare la massima frequenza di trasferimento dati ottenibile (espressa in kB/s) fra la periferica e la memoria principale.

65% NON I/O

35% I/O

1 PAROLA = 4 ISTRUZIONI = 32 bit = 4 B

1 ISTRUZIONE = $\frac{1}{4}$ PAROLA

$$0,35 \cdot 10^5 \text{ istruzioni/s} = \frac{0,35}{1} \cdot 10^5 \text{ PAROLA/s} = \frac{0,35}{1} \cdot 10^5 \cdot 4 \text{ B}$$

34,18 KB/s

10:14 Mar 4 Giu 1 minuto fa

eseguire programmi che non contengono trasferimenti di I/O. l'ampiezza della linea dati del bus è pari a 32 bit. Si consideri il caso in cui il trasferimento di una parola da 32 bit fra una periferica e la memoria principale avvenga mediante I/O da programma per mezzo delle seguenti 4 istruzioni:

- LOAD parola dalla periferica al registro CPU
- STORE parola da registro CPU a memoria
- generazione indirizzo di memoria successivo
- conteggio dati da trasferire.

Calcolare la massima frequenza di trasferimento dati ottenibile (espressa in kB/s) fra la periferica e la memoria principale.

Soluzione

Per trasferire una parola da 32 bit occorrono 4 istruzioni mediante I/O da programma. La CPU è impegnata per l'65% del tempo a eseguire istruzioni che non coinvolgono l'I/O, dunque può usare solo il 35% del tempo per eseguire istruzioni di trasferimento dati con periferiche. In termini di istr./sec questo tempo è pari a $0.35 \times 10^6 \text{ istr./s} = 3.5 \times 10^4 \text{ istr./s}$. Dal momento che per trasferire una parola servono quattro istruzioni, la velocità di trasferimento è pari a: $3.5 \times 10^4 \text{ istr./s} / (4 \text{ istr./parola}) = 8750 \text{ parole/s}$. La dimensione di una parola è pari a 32 bit (4 byte), da cui si ricava una velocità di trasferimento di circa **34.18 kB/s**.

ESERCIZIO 2 PARTE PRIMA

Scrivere il codice Assembly che assegni il valore 3 alla parola di memoria all'indirizzo $2^{16}+5$. Si usino i registri che si ritengono opportuni.

```

ASRDi $2, $0, 3      # $2 -> 3
LUI    $1, 1          # CARICA bit più significativi 2^6 $1 -> 2^6
ADDi   $1, $1, 5       # AGGIUNGI 5 -> $1
SW     $2, 0($1)       # ASSEGNARE IL VALORE 3 IN $2 IN $1
  
```

OPPURE

```

MOVE $8, $0          # INIZIAZZO $8
  
```

```

LUI    $8, 1           # $8 ← 216
ADDI   $8, $8, 5       # $8 ← 216 + 5
ADDI   $9, $0, 3       # 9 ← 3
SW     $9, 0($8)       # SALVO IL VALORE 3 ALL'INDIRIZZO
                           PRESENTE IN $8

```

ESERCIZIO 3 PRIMA PARTE

Scrivere un frammento di codice Assembly MIPS che implementi l'istruzione: $A[i] = A[i+4] + A[i-2]$.

Si consideri A come indirizzo simbolico ed il valore di i sia memorizzato nel registro \$3.

```

MULI   $8, $3, 4       # $8 ← i * 4
ADDI   $9, $8, 16      # $9 ← (i + 4) * 4
LW     $11, A($9)      # $11 ← A[i + 4]
ADDI   $9, $8, -8      # $9 ← (i - 2) * 4
LW     $12, A($9)      # $12 ← A[i - 2]
ADDI   $12, $11, $12    # $12 ← A[i + 4] + A[i - 2]
SW     $12, A($8)      # $12 → A[i]

```

ESERCIZIO 1 SECONDA PARTE

Scrivere una funzione Assembly MIPS moltiplica che implementi il prodotto tra due numeri naturali forniti nei registri \$4 e \$5, senza far uso dell'istruzione mul o muli. Il risultato dell'operazione venga memorizzato in \$7.

```

INT moltiplica (INT A, INT B)
{
    INT i = 1;
    INT r = 0;
    WHILE (i <= b);

```

CODICE C

```

{
    r+=a;
    i++;
}
RETURN(r);
}

```



CODICE ASSEMBLY

MULTIPLICA:	ADDI \$29, \$29, -4	# SALVATAGGIO DEL
	SW \$8, 0(\$29)	CONTENUTO
	MOVE \$7, \$0	# \$7 ← 0
	MOVE \$8, \$0	# \$8 ← i = 0
LOOP:	BGE \$8, \$5, EXIT	# IF i == \$5 VAI EXIT
	ADDI \$7, \$7, \$4	# \$7 ← \$7 + \$4
	ADDI \$8, \$8, 1	# i++
	J LOOP	
EXIT:	LW \$8, 0(\$29)	
	ADDI \$29, \$29, 4	# RIPRISTINO DEL
	JR \$31	CONTESTO
		# RITORNO AL CHIAMANTE

ESERCIZIO 3 SECONDA PARTE

Si consideri la seguente istruzione Assembly MIPS espressa in codice binario

LW |----- REGISTRI -----| 1200

10011

10011

01000

0000 0100 1011 0000

Con l'aiuto delle slide fornite a lezione (p. 44 e seguenti), indicare a quale possibile istruzione essa corrisponde motivando la ragione.

LW \$8, 1200 (\$19)

} OPPURE

✓
LW \$8, V (\$19)

